Doc. No.:	P0096R4
Date:	2017-07-26
Reply to:	Clark Nelson
Title:	Feature-testing recommendations for C++

Feature-testing recommendations for C++

[-1] Preface

[1] ~~*This revision of this document contains STUBS for sections expected to be filled in later.*~~

[0] Contents

[1] Introduction

[1] At the September 2013 (Chicago) meeting of WG21, there was a five-way poll of all of the C++ experts in attendance – approximately 80 – concerning their support for the approach described herein for feature-testing in C++. The results of the poll:

Strongly favor	Favor	Neutral	Oppose	Strongly oppose
lots	lots	1	0	0

[2] General support for these recommendations was reaffirmed at the 2015 October (Kona) meeting, as indicated by the following straw poll:

Should WG21 encourage, but not require, feature-test macros for proposals?

Strongly favor	Favor	Neutral	Against	Strongly against
50	13	7	3	2

[3] This document was subsequently designated WG21's SD-6 (sixth standing document), which will continue to be maintained by SG10.

[2] Explanation and rationale for the approach

[2.1] Problem statement

[1] The pace of innovation in the standardization of C++ makes long-term stability of implementations unlikely. Features are added to the language because programmers want to use those features. Features are added to (the working draft of) the standard as the features become well-specified. In many cases a feature is added to an implementation well before or well after the standard officially introducing it is approved.

[2] This process makes it difficult for programmers who want to use a feature to know whether it is available in any given implementation. Implementations rarely leap from one formal revision of the standard directly to the next; the implementation process generally proceeds by smaller steps. As a result, testing for a specific revision of the standard (e.g. by examining the value of the `__cplusplus` macro) often gives the wrong answer. Implementers generally don't want to appear to be claiming full conformance to a standard revision until all of its features are implemented. That leaves programmers with no portable way to determine which features are actually available to them.

[3] It is often possible for a program to determine, in a manner specific to a single implementation, what features are supported by that implementation; but the means are often poorly documented and ad hoc, and sometimes complex – especially when the availability of a feature is controlled by an invocation option. To make this determination for a variety of implementations in a single source base is complex and error-prone.

[2.2] Status quo

[1] Here is some code that attempts to determine whether rvalue references are available in the implementation in use:

```
#ifndef __USE_RVALUE_REFERENCES
  #if (__GNUC__ > 4 || __GNUC__ == 4 && __GNUC_MINOR__ >= 3) || \
      _MSC_VER >= 1600
    #if __EDG_VERSION__ > 0
      #define __USE_RVALUE_REFERENCES (__EDG_VERSION__ >= 410)
    #else
      #define __USE_RVALUE_REFERENCES 1
    #endif
  #elif __clang__
    #define __USE_RVALUE_REFERENCES __has_feature(cxx_rvalue_references)
  #else
    #define __USE_RVALUE_REFERENCES 0
  #endif
#endif
```

[2] First, the GNU and Microsoft version numbers are checked to see if they are high enough. But then a check is made of the EDG version number, since that front end also has compatibility modes for both those compilers, and defines macros indicating (claimed) compatibility with them. If the feature wasn't implemented in the indicated EDG version, it is assumed that the feature is not available – even though it is possible for a customer of EDG to implement a feature before EDG does.

[3] Fortunately Clang has ways to test specifically for the presence of specific features. But unfortunately, the function-call-like syntax used for such tests won't work with a standard preprocessor, so this fine new feature winds up adding its own flavor of complexity to the mix.

[4] Also note that this code is only the beginning of a real-world solution. A complete solution would need to take into account more compilers, and also command-line option settings specific to various compilers.

[2.3] Characteristics of the proposed solution

[1] To preserve implementers' freedom to add features in the order that makes the most sense for themselves and their customers, implementers should indicate the availability of each separate feature by adding a definition of a macro with the name corresponding to that feature.

[2] **Important note:** By recommending the use of these macros, WG21 is **not** making any feature optional; the absence of a definition for the relevant feature-test macro does not make an implementation that lacks a feature conform to a standard that requires the feature. However, if implementers and programmers follow these recommendations, portability of code between real-world implementations should be improved.

[3] To a first approximation, a feature is identified by the WG21 paper in which it is specified, and by which it is introduced into the working draft of the standard. Not every paper introduces a new feature worth a feature-test macro, but every paper that is not just a collection of issue resolutions is considered a candidate; exceptions are explicitly justified.

[4] For C++14, the feature-test macro name generally consists of some combination of words from the title of the paper. In the future, it is hoped that every paper will include its own recommendations concerning feature-test macro names.

[5] The value specified for a feature-test macro is based on the year and month in which the feature is voted into the working draft. In a case where a feature is subsequently changed in a significant way, but arguably remains the same feature, the value of the macro is changed to indicate the "revision level" of the specification of the feature. However, in most cases it is expected that the presence of a feature can be determined by the presence of any non-zero macro value; for example:

```
#if __cpp_binary_literals
int const packed_zero_to_three = 0b00011011;
#else
int const packed_zero_to_three = 0x1B;
#endif
```

[6] To avoid the user's namespace, names of macros for language features are prefixed by "`__cpp_`"; for library features, by "`__cpp_lib_`". A library feature that doesn't introduce a new header is expected to be defined by the header(s) that implement the feature.

[3] Recommendations

[3.1] Introduction

[1] For the sake of improved portability between partial implementations of various C++ standards, WG21 (the ISO technical committee for the C++ programming language) recommends that implementers and programmers follow the guidelines in this document concerning feature-test macros.

[2] Implementers who provide a new standard feature should define a macro with the recommended name and value, in the same circumstances under which the feature is available (for example, taking into account relevant command-line options), to indicate the presence of support for that feature.

[3] Programmers who wish to determine whether a feature is available in an implementation should base that determination on the state of the macro with the recommended name. (The absence of a tested feature may result in a program with decreased functionality, or the relevant functionality may be provided in a different way. A program that strictly depends on support for a feature can just try to use the feature unconditionally; presumably, on an implementation lacking necessary support, translation will fail. Therefore, if the most useful purpose for a feature-test macro would be to control the inclusion of a `#error` directive if the feature is unavailable, that is considered inadequate justification for the macro. Note that the usefulness of a test macro for a feature is completely independent of the usefulness of the feature itself.)

[3.2] Testing for the presence of a header: `__has_include`

[1] It is impossible for a C++ program to directly, reliably and portably determine whether or not a library header is available for inclusion. Conditionally including a header requires the use of a configuration macro, whose setting can be determined by a configuration-test process at build

time (reliable, but less portable), or by some other means (often not reliable or portable).

[2] To solve this general problem, WG21 recommends that implementers provide, and programmers use, the `__has_include` feature.

[3.2.1] Syntax

h-preprocessing-token:
any *preprocessing-token* other than `>`

h-pp-tokens:
h-preprocessing-token
h-pp-tokens h-preprocessing-token

has-include-expression:
`__has_include (` *header-name* `)`
`__has_include (` *string-literal* `)`
`__has_include ( <` *h-pp-tokens* `> )`

[3.2.2] Semantics

[1] In the first form of the *has-include-expression*, the parenthesized *header-name* token is not subject to macro expansion. The second and third forms are considered only if the first form does not match, and the preprocessing tokens are processed just as in normal text.

[2] A *has-include-expression* shall appear only in the controlling constant expression of a `#if` or `#elif` directive ([cpp.cond] 16.1). Prior to the evaluation of such an expression, the source file identified by the parenthesized preprocessing token sequence in each contained *has-include-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* in a `#include` directive, except that no further macro expansion is performed. If such a directive would not satisfy the syntactic requirements of a `#include` directive, the program is ill-formed. The *has-include-expression* is replaced by the *pp-number* `1` if the search for the source file succeeds, and by the *pp-number* `0` if the search fails.

[3] The `#ifdef` and `#ifndef` directives, and the `defined` conditional inclusion operator, shall treat `__has_include` as if it were the name of a defined macro. The identifier `__has_include` shall not appear in any context not mentioned in this section.

[3.2.3] Example

[1] This demonstrates a way to use a library `optional` facility only if it is available.

```
#ifdef __has_include
#  if __has_include(<optional>)
#    include <optional>
#    define have_optional 1
#  elif __has_include(<experimental/optional>)
#    include <experimental/optional>
#    define have_optional 1
#    define experimental_optional
#  else
#    define have_optional 0
#  endif
#endif
```

[3.3] Testing for the presence of an attribute: `__has_cpp_attribute`

[1] A C++ program cannot directly, reliably, and portably determine whether or not a standard or vendor-specific attribute is available for use. Testing for attribute support generally requires complex macro logic, as illustrated above for language features in general.

[2] To solve this general problem, WG21 recommends that implementers provide, and programmers use, the `__has_cpp_attribute` feature.

[3.3.1] Syntax

has-attribute-expression:
`__has_cpp_attribute (` *attribute-token* `)`

[3.3.2] Semantics

[1] A *has-attribute-expression* shall appear only in the controlling constant expression of a `#if` or `#elif` directive ([cpp.cond] 16.1). The *has-attribute-expression* is replaced by a non-zero pp-number if the implementation supports an attribute with the specified name, and by the pp-number 0 otherwise.

[2] For a standard attribute, the value of the `__has_cpp_attribute` macro is based on the year and month in which the attribute was voted into the working draft. In the case where the attribute is vendor-specific, the value is implementation-defined. However, in most cases it is expected that the availability of an attribute can be detected by any non-zero result.

[3] The `#ifdef` and `#ifndef` directives, and the `defined` conditional inclusion operator, shall treat `__has_cpp_attribute` as if it were the name of a defined macro. The identifier `__has_cpp_attribute` shall not appear in any context not mentioned in this section.

[3.3.3] Example

[1] This demonstrates a way to use the attribute `[[deprecated]]` only if it is available.

```
#ifndef __has_cpp_attribute
```

```
# define __has_cpp_attribute(x) 0
#endif
#ifdef __has_cpp_attribute
#if __has_cpp_attribute(deprecated)
# define ATTR_DEPRECATED(msg) [[deprecated(msg)]]
#else
# define ATTR_DEPRECATED(msg)
#endif
#endif
```

[3.4] C++17 features

[1] The following table itemizes all the changes that were made to the working draft for C++17 as specified in a WG21 technical document. (Changes that were made as specified in a core or library issue are not generally included.)

[2] The table is sorted by the section of the standard primarily affected. The “Doc. No.” column links to the paper itself on the committee web site. The “Macro Name” column links to the relevant portion of the “Detailed explanation and rationale” section of this document. When the recommendation is to change the value of a macro previously recommended to be defined, the “Value” column links to the table entry for the previous recommendation.

[3] For library features, the “Header“ column identifies the header that is expected to define the macro, although the macro may also be predefined. For language features, the macro must be predefined.

Significant changes to C++17

Doc. No.	Title	Primary Section	Macro Name	Value	Header
P0296R2	Forward progress guarantees: Base definitions	1.10	*none*		
P0299R1	Forward progress guarantees for the Parallelism TS features	1.10, 25.2	*none*		
N4086	Removing trigraphs??!	2.4	*none*		
P0245R1	Hexadecimal floating literals for C++	2.13	`__cpp_hex_float` ex.	201603	*predefined*
N4267	Adding u8 character literals	2.14	*none*		
P0386R2	Inline Variables	3.6, 7.1	`__cpp_inline_variables` ex.	201606	*predefined*
P0035R4	Dynamic memory allocation for over-aligned data	3.7, 5.3, 18.6	`__cpp_aligned_new` ex.	201606	*predefined*
P0135R1	Wording for guaranteed copy elision through simplified value categories	3.10	*none*		
N4261	Proposed resolution for Core Issue 330: Qualification conversions and pointers to arrays of pointers	4.4, 5.2	*none*		
P0012R1	Make exception specifications be part of the type system	4.12, 15.4	`__cpp_noexcept_function_type` ex.	201510	*predefined*
P0145R3	Refining Expression Evaluation Order for Idiomatic C++	5	*none*		
N4295	Folding expressions	5.1, 14.5, 14.6	`__cpp_fold_expressions`	201411	*predefined*
P0018R3	Lambda Capture of *this by Value as [=,*this]	5.1	`__cpp_capture_star_this` ex.	201603	*predefined*
P0170R1	Wording for Constexpr Lambda	5.1	`__cpp_constexpr`	201603	*predefined*
P0002R1	Remove Deprecated operator++(bool)	5.3	*none*		
P0292R2	constexpr if: A slightly different syntax	6.4	`__cpp_if_constexpr` ex.	201606	*predefined*
P0305R1	Selection statements with initializer	6.4	*none*		
P0184R0	Generalizing the Range-Based For Loop	6.5	`__cpp_range_based_for`	201603	*predefined*
N3928	Extending static_assert	7	`__cpp_static_assert`	201411	*predefined*
N3922	New Rules for auto deduction from braced-init-list	7.1	*none*		
P0001R1	Remove Deprecated Use of the register Keyword	7.1	*none*		
P0091R3	Template argument deduction for class templates	7.1, 13.3, 14.9	`__cpp_deduction_guides` ex.	201606	*predefined*
P0512R0	Class Template Argument Deduction Assorted NB resolution and issues	13.3, 14.9	`__cpp_deduction_guides` ex.	201611	*predefined*
P0127R2	Declaring non-type template parameters with auto	7.1, 14.8	`__cpp_template_auto` `__cpp_nontype_template_argument_auto` ex.	201606	*predefined*
N4266	Attributes for namespaces and enumerators	7.2, 7.3	`__cpp_namespace_attributes`	201411	*predefined*
			`__cpp_enumerator_attributes`	201411	*predefined*
N4230	Nested namespace definition	7.3	`__cpp_nested_namespace_definitions` ex. *none*	201411	*predefined*
P0136R1	Rewording inheriting constructors (core issue 1941 et al)	7.3	`__cpp_inheriting_constructors` ex.	201511	*predefined*
P0195R2	Pack expansions in using-declarations	7.3	`__cpp_variadic_using`	201611	*predefined*
P0188R1	Wording for [[fallthrough]] attribute	7.6	`__has_cpp_attribute(fallthrough)`	201603	*predefined*
P0189R1	Wording for [[nodiscard]] attribute	7.6	`__has_cpp_attribute(nodiscard)`	201603	*predefined*

P0212R1	Wording for [[maybe_unused]] attribute	7.6	__has_cpp_attribute(maybe_unused)	201603	*predefined*
P0028R4	Using attribute namespaces without repetition	7.6	???	201606	*predefined*
P0283R2	Standard and non-standard attributes	7.6	???	201606	*predefined*
P0017R1	Extension to aggregate initialization	8.5	__cpp_aggregate_bases ex.	201603	*predefined*
P0138R2	Construction Rules for enum class Values	8.5	*none*		
P0217R3	Proposed wording for structured bindings	8.5	__cpp_structured_bindings ex.	201606	*predefined*
P0398R0	Core issue 1518: Explicit default constructors and copy-list-initialization	8.5	*none*		
P0134R0	Introducing a name for brace-or-equal-initializers for non-static data members	9.2	*none*		
P0391R0	Introducing the term "templated entity"	14	*none*		
N4051	Allow typename in a template template parameter	14.1	*none*		
N4268	Allow constant evaluation for all non-type template arguments	14.3	__cpp_nontype_template_args	201411	*predefined*
P0522R0	Matching of template template-arguments excludes compatible templates	14.3	__cpp_template_template_args ex.	201611	*predefined*
P0036R0	Unary Folds and Empty Parameter Packs	14.5	__cpp_fold_expressions	201603	*predefined*
N4262	Wording for Forwarding References	14.8	*none*		
N4285	Cleanup for exception-specification and throw-expression	15	*none*		
P0003R5	Removing Deprecated Exception Specifications from C++17	15.4	???		
P0061R1	__has_include for C++17	16.1	__has_include	*defined*	*predefined*
P0063R3	C++17 should refer to C11 instead of C99	17.6	*none*		
P0180R2	Reserve a New Library Namespace Future Standardization	17.6	*none*		
P0175R1	Synopses for the C library	18 etc.	*none*		
P0298R3	A byte type definition	18.2	__cpp_lib_byte ex.	201603	<cstddef>
P0154R1	constexpr std::hardware_{constructive,destructive}_interference_size	18.6	__cpp_lib_hardware_interference_size ex.	201703	<new>
P0137R1	Core Issue 1776: Replacement of class objects containing reference members	18.6, 1.8	__cpp_lib_launder ex.	201606	<new>
N4259	Wording for std::uncaught_exceptions	18.8	__cpp_lib_uncaught_exceptions	201411	<exception>
P0067R5	Elementary string conversions	20	__cpp_lib_to_chars ex.	201611	<utility>
P0220R1	Adopt Library Fundamentals V1 TS Components for C++17	20	__has_include(<optional>)	1	*predefined*
			__cpp_lib_optional	201603	<optional>
			__has_include(<any>)	1	*predefined*
			__cpp_lib_any	201603	<any>
			__has_include(<string_view>)	1	*predefined*
			__cpp_lib_string_view	201603	<string_view>
			__has_include(<memory_resource>)	1	*predefined*
			__cpp_lib_memory_resource	201603	<memory_resource>
		20.5	__cpp_lib_apply ex.	201603	<tuple>
		20.11	__cpp_lib_shared_ptr_arrays edit fail	201603	<memory>
		20.14	__cpp_lib_boyer_moore_searcher ex.	201603	<functional>
		25.4	__cpp_lib_sample ex.	201603	<algorithm>
P0007R1	Constant View: A proposal for a std::as_const helper function template	20.2	__cpp_lib_as_const ex.	201510	<utility>
P0032R3	Homogeneous interface for variant, any and optional	20.2, 20.6-8	*none* ?		
N4387	Improving pair and tuple	20.4, 20.5	*none* ???	201505	<utility> <tuple>
P0209R2	make_from_tuple: apply for construction	20.5	__cpp_lib_make_from_tuple ex.	201606	<tuple>
P0307R2	Making Optional Greater Equal Again	20.5	???	201606	<optional>
P0088R3	Variant: a type-safe union for C++17	20.7	__has_include(<variant>)	1	*predefined*
			__cpp_lib_variant ex.	201606	<variant>
P0393R3	Making Variant Greater Equal	20.7	*none*		
LWG2296	std::addressof should be constexpr	20.10	__cpp_lib_addressof_constexpr	201603	<memory>
P0040R3	Extending memory management tools	20.10	???	201606	<memory>
P0174R2	Deprecating Vestigial Library Parts in C++17	20.10, 24.4	*none*		
N4190	Removing auto_ptr, random_shuffle(), And Old <functional> Stuff	20.11-20.14	*none*		

P0074R0	Making std::owner_less more flexible	20.11	__cpp_lib_transparent_operators	201510	<memory> <functional> ?
N4089	Safe conversions in unique_ptr<T[]>	20.11	*none*		
N4366	LWG 2228: Missing SFINAE rule in unique_ptr templated assignment	20.11	*none*		
P0033R1	Re-enabling shared_from_this	20.11	__cpp_lib_enable_shared_from_this	201603	<memory>
P0163R0	shared_ptr::weak_type	20.11	__cpp_lib_shared_ptr_weak_type	201606	<memory>
P0497R0	Fixes to shared_ptr support for arrays	20.11	__cpp_lib_shared_ptr_arrays ex.	201611	<memory>
P0337R0	Delete operator= for polymorphic_allocator	20.12	*none*		
N4169	A proposal to add invoke function template	20.14	__cpp_lib_invoke	201411	<functional>
N4277	TriviallyCopyable reference_wrapper	20.14	*none*		
P0005R4	Adopt not_fn from Library Fundamentals 2 for C++17	20.14	__cpp_lib_not_fn ex.	201603	<functional>
P0358R1	Fixes for not_fn	20.14	*none*		
P0253R1	Fixing a design mistake in the searchers interface in Library Fundamentals	20.14	*none*		
P0302R1	Removing Allocator Support in std::function	20.14	*none*		
N3911	TransformationTrait Alias void_t	20.15	__cpp_lib_void_t	201411	<type_traits>
N4389	Wording for bool_constant	20.15	__cpp_lib_bool_constant	201505	<type_traits>
P0006R0	Adopt Type Traits Variable Templates from Library Fundamentals TS for C++17	20.15	__cpp_lib_type_trait_variable_templates ex.	201510	<type_traits>
P0013R1	Logical Operator Type Traits	20.15	__cpp_lib_logical_traits	201510	<type_traits>
P0185R1	Adding [nothrow-]swappable traits	20.15	__cpp_lib_is_swappable	201603	<type_traits>
P0604R0	Resolving GB 55, US 84, US 85, US 86	20.15	__cpp_lib_is_invocable ex.	201703	<type_traits>
P0258R2	has_unique_object_representations - wording	20.15	__cpp_lib_has_unique_object_representations ex.	201606	<type_traits>
LWG2911	An is_aggregate type trait is needed	20.15	__cpp_lib_is_aggregate ex.	201703	<type_traits>
P0092R1	Polishing <chrono>	20.17	__cpp_lib_chrono ex.	201510	<chrono>
P0505R0	Wording for GB 50	20.17	__cpp_lib_chrono ex.	201611	<chrono>
P0336R1	Better Names for Parallel Execution Policies in C++17	20.19	???	201606	<execution>
P0254R2	Integrating std::string_view and std::string	21.3, 21.4	???	201606	<string> <string_view>
P0272R1	Give 'std::string' a non-const '.data()' member function	21.4	*none*		
N4258	Cleaning-up noexcept in the Library	21.4, 23.3-23.5	__cpp_lib_allocator_traits_is_always_equal	201411	<memory> <scoped_allocator> <string> <deque> <forward_list> <list> <vector> <map> <set> <unordered_map> <unordered_set>
P0618R0	Deprecating <codecvt>	22.5	*none* ?		
N4284	Contiguous Iterators	23.2, 24.2	*none*		
N4510	Minimal incomplete type support for standard containers	23.3	__cpp_lib_incomplete_container_elements ex.	201505	*headers*
P0084R2	Emplace Return Type	23.3	*none*		
N4279	Improved insertion interface for unique-key maps	23.4	__cpp_lib_map_try_emplace	201411	<map>
		23.5	__cpp_lib_unordered_map_try_emplace	201411	<unordered_map>
P0083R3	Splicing Maps and Sets	23.3	__cpp_lib_node_extract	201606	<map> <set> <unordered_map> <unordered_set>
N4280	Non-member size() and more	24.3	__cpp_lib_nonmember_container_access	201411	<iterator> <array> <deque> <forward_list> <list> <map> <regex> <set> <string> <unordered_map> <unordered_set> <vector>
P0031R0	A Proposal to Add Constexpr Modifiers to reverse_iterator, move_iterator, array and Range Access	24.3	__cpp_lib_array_constexpr ex.	201603	<iterator> <array>

P0024R2	The Parallelism TS Should be Standardized	20	__has_include(<execution>)	1	*predefined*
			__lib_cpp_execution ?	201603	<execution>
		25, 26	__cpp_lib_parallel_algorithm ex.	201603	<algorithm> <numeric>
P0394R4	Hotel Parallelifornia: terminate() for Parallel Algorithms Exception Handling	25.2, 18.8	???	201606	<execution>
P0025R0	An algorithm to "clamp" a value between a pair of boundary values	25.4	__cpp_lib_clamp ex.	201603	<algorithm>
P0346R1	A <random> Nomenclature Tweak	26.6	*none*		
P0295R0	Adopt Selected Library Fundamentals V2 Components for C++17	26.8	__cpp_lib_gcd __cpp_lib_lcm ex.	201606	<numeric>
P0030R1	Proposal to Introduce a 3-Argument Overload to std::hypot	26.9	__cpp_lib_hypot ex.	201603	<cmath>
P0226R1	Mathematical Special Functions for C++17	26.10	__cpp_lib_math_special_functions ex.	201603	<cmath>
P0218R1	Adopt the File System TS for C++17	27.10	__has_include(<filesystem>)	1	*predefined*
			__cpp_lib_filesystem ex.	201603	<filesystem>
P0219R1	Relative Paths for Filesystem	27.10	???	201606	<filesystem>
P0317R1	Directory Entry Caching for Filesystem	27.10	???	201703	<filesystem>
P0392R0	Adapting string_view by filesystem paths	27.10	???	201606	<filesystem>
P0371R1	Temporarily discourage memory_order_consume	29.3	*none*		
P0152R1	constexpr atomic<T>::is_always_lock_free	29.5	__cpp_lib_atomic_is_always_lock_free ex.	201603	<atomic>
N4508	A proposal to add shared_mutex (untimed)	30.4	__cpp_lib_shared_mutex ex.	201505	<shared_mutex>
~~P0156R0~~	~~Variadic lock_guard~~	~~30.4~~	~~__cpp_lib_lock_guard_variadic~~	~~201510~~	~~<thread>~~
P0156R2	Variadic lock_guard (Rev. 5)	30.4	__cpp_lib_scoped_lock ex.	201703	<mutex>
P0004R1	Remove Deprecated iostreams aliases	D	*none*		

[3.5] C++14 features

[1] The following table itemizes all the changes that were made to the working draft for C++14 as specified in a WG21 technical document. (Changes that were made as specified in a core or library issue are not generally included.)

[2] The table is sorted by the section of the standard primarily affected. The “Doc. No.” column links to the paper itself on the committee web site. The “Macro Name” column links to the relevant portion of the “Detailed explanation and rationale” section of this document. When the recommendation is to change the value of a macro previously recommended to be defined, the “Value” column links to the table entry for the previous recommendation.

[3] For library features, the “Header“ column identifies the header that is expected to define the macro, although the macro may also be predefined. For language features, the macro must be predefined.

Significant changes to C++14

Doc. No.	Title	Primary Section	Macro Name	Value	Header
N3910	What can signal handlers do? (CWG 1441)	1.9-1.10	*none*		
N3927	Definition of Lock-Free	1.10	*none*		
N3472	Binary Literals in the C++ Core Language	2.14	__cpp_binary_literals	201304	*predefined*
N3781	Single-Quotation-Mark as a Digit Separator	2.14	*none*		
N3323	A Proposal to Tweak Certain C++ Contextual Conversions	4	*none*		
N3648	Wording Changes for Generalized Lambda-capture	5.1	__cpp_init_captures	201304	*predefined*
N3649	Generic (Polymorphic) Lambda Expressions	5.1	__cpp_generic_lambdas	201304	*predefined*
N3664	Clarifying Memory Allocation	5.3	*none*		
N3778	C++ Sized Deallocation	5.3, 18.6	__cpp_sized_deallocation	201309	*predefined*
N3624	Core Issue 1512: Pointer comparison vs qualification conversions	5.9, 5.10	*none*		
N3652	Relaxing constraints on constexpr functions / constexpr member functions and implicit const	5.19, 7.1	__cpp_constexpr	201304	*predefined*
N3638	Return type deduction for normal functions	7.1	__cpp_decltype_auto	201304	*predefined*
			__cpp_return_type_deduction	201304	*predefined*
N3760	[[deprecated]] attribute	7.6	__has_cpp_attribute(deprecated)	201309	*predefined*
N3653	Member initializers and aggregates	8.5	__cpp_aggregate_nsdmi	201304	*predefined*
N3667	Drafting for Core 1402	12.8	*none*		
N3651	Variable Templates	14, 14.7	__cpp_variable_templates	201304	*predefined*
N3669	Fixing constexpr member functions without const	*various*	*none*		
N3673	C++ Library Working Group Ready Issues Bristol 2013	*various*	*none*		
N3658	Compile-time integer sequences	20	__cpp_lib_integer_sequence	201304	<utility>

N3668	exchange() utility function	20	__cpp_lib_exchange_function	201304	<utility>
N3471	Constexpr Library Additions: utilities	20.2-20.4	*none*		
N3670	Wording for Addressing Tuples by Type	20.2-20.4	__cpp_lib_tuples_by_type	201304	<utility>
N3887	Consistent Metafunction Aliases	20.3-20.4	__cpp_lib_tuple_element_t	201402	<utility>
N3656	make_unique	20.7	__cpp_lib_make_unique	201304	<memory>
N3421	Making Operator Functors greater<>	20.8	__cpp_lib_transparent_operators	201210	<functional>
N3545	An Incremental Improvement to integral_constant	20.9	__cpp_lib_integral_constant_callable	201304	<type_traits>
N3655	TransformationTraits Redux	20.9	__cpp_lib_transformation_trait_aliases	201304	<type_traits>
N3789	Constexpr Library Additions: functional	20.10	*none*		
N3462	std::result_of and SFINAE	20.9 20.10	__cpp_lib_result_of_sfinae	201210	<functional> <type_traits>
LWG 2112	User-defined classes that cannot be derived from	20.10	__cpp_lib_is_final	201402	<type_traits>
LWG 2247	Type traits and std::nullptr_t	20.10	__cpp_lib_is_null_pointer	201309	<type_traits>
N3469	Constexpr Library Additions: chrono	20.11	*none*		
N3642	User-defined Literals for Standard Library Types	20.11	__cpp_lib_chrono_udls	201304	<chrono>
		21.7	__cpp_lib_string_udls	201304	<string>
N3470	Constexpr Library Additions: containers	23.3	*none*		
N3657	Adding heterogeneous comparison lookup to associative containers	23.4	__cpp_lib_generic_associative_lookup	201304	<map> <set>
N3644	Null Forward Iterators	24.2	__cpp_lib_null_iterators	201304	<iterator>
LWG 2285	make_reverse_iterator	24.5	__cpp_lib_make_reverse_iterator	201402	<iterator>
N3671	Making non-modifying sequence operations more robust	25.2	__cpp_lib_robust_nonmodifying_seq_ops	201304	<algorithm>
N3779	User-defined Literals for std::complex	26.4	__cpp_lib_complex_udls	201309	<complex>
N3924	Discouraging rand() in C++14	26.8	*none*		
N3654	Quoted Strings Library Proposal	27.7	__cpp_lib_quoted_string_io	201304	<iomanip>
LWG 2249	Remove gets from <cstdio>	27.9	*none*		
N3786	Prohibiting "out of thin air" results in C++14	29.3	*none*		
N3659	Shared locking in C++	30.4	__has_include(<shared_mutex>)	1	*predefined*
N3891	A proposal to rename shared_mutex to shared_timed_mutex	30.4	__cpp_lib_shared_timed_mutex	201402	<shared_mutex>
N3776	Wording for ~future	30.6	*none*		

[3.6] C++11 features

Significant features of C++11

Doc. No.	Title	Primary Section	Macro name	Value	Header
N2249	New Character Types in C++	2.13	__cpp_unicode_characters	200704	*predefined*
N2442	Raw and Unicode String Literals Unified Proposal	2.13	__cpp_raw_strings	200710	*predefined*
			__cpp_unicode_literals	200710	*predefined*
N2765	User-defined Literals	2.13, 13.5	__cpp_user_defined_literals	200809	*predefined*
N2660	Dynamic Initialization and Destruction with Concurrency	3.6	__cpp_threadsafe_static_init	200806	*predefined*
N2927	New wording for C++0x lambdas	5.1	__cpp_lambdas	200907	*predefined*
N2235	Generalized Constant Expressions	5.19, 7.1	__cpp_constexpr	200704	*predefined*
N2930	Range-Based For Loop Wording (Without Concepts)	6.5	__cpp_range_based_for	200907	*predefined*
N1720	Proposal to Add Static Assertions to the Core Language	7	__cpp_static_assert	200410	*predefined*
N2343	Decltype	7.1	__cpp_decltype	200707	*predefined*
N2761	Towards support for attributes in C++	7.6	__cpp_attributes	200809	*predefined*
			__has_cpp_attribute(noreturn)	200809	*predefined*
			__has_cpp_attribute(carries_dependency)	200809	*predefined*
N2118	A Proposal to Add an Rvalue Reference to the C++ Language	8.3	__cpp_rvalue_references	200610	*predefined*
N2242	Proposed Wording for Variadic Templates	8.3, 14	__cpp_variadic_templates	200704	*predefined*
N2672	Initializer List proposed wording	8.5	__cpp_initializer_lists	200806	*predefined*
N1986	Delegating Constructors	12.6	__cpp_delegating_constructors	200604	*predefined*
N2756	Non-static data member initializers	12.6	__cpp_nsdmi	200809	*predefined*
N2540	Inheriting Constructors	12.9	__cpp_inheriting_constructors	200802	*predefined*
N2439	Extending move semantics to *this	13.3	__cpp_ref_qualifiers	200710	*predefined*

N2258	Template Aliases	14.5	__cpp_alias_templates	200704	*predefined*

[3.7] ~~Conditionally-supported constructs~~

[1] ~~*STUB*~~

[3.8] C++98 features

[1] ~~*STUB: especially for exception handling and RTTI*~~

[2] With very few exceptions, the features of C++98 have all been implemented in virtually every C++ compiler. But in many compilers, some of them can be enabled/disabled. It is recommended that the macros in the following table should be used to indicate whether one of these features is enabled in the current compilation.

Feature	Primary section	Macro name	Value	Header
Run-time type identification	5.2	__cpp_rtti	199711	*predefined*
Exception handling	15	__cpp_exceptions	199711	*predefined*

[3.9] Features published and later removed

[3.9.1] Features removed from C++17

Features in the CD of C++17, subsequently removed

Doc. No.	Title	Primary Section	Macro Name	Value	Header
P0077R2	is_callable, the missing INVOKE related trait	20.15	__cpp_lib_is_callable	201603	<type_traits>
P0181R1	Ordered By Default	20.14, 23.4, 23.6	__cpp_lib_default_order	201606	<type_traits>
P0156R0	Variadic lock_guard	30.4	__cpp_lib_lock_guard_variadic	201510	~~<thread>~~ <mutex>

[3.9.2] Features removed from C++14

Features in the first CD of C++14, subsequently removed to a Technical Specification

Doc. No.	Title	Primary Section	Macro Name	Value	Header
N3639	Runtime-sized arrays with automatic storage duration	8.3	__cpp_runtime_arrays	201304	*predefined*
N3672	A proposal to add a utility class to represent optional objects	20.5	__has_include(<optional>)	1	*predefined*
N3662	C++ Dynamic Arrays	23.2, 23.3	__has_include(<dynarray>)	1	*predefined*

[1] The intention is that an implementation which provides runtime-sized arrays as specified in the first CD should define __cpp_runtime_arrays as 201304. The expectation is that a later document specifying runtime-sized arrays will specify a different value for __cpp_runtime_arrays.

[4] Recommendations from Technical Specifications

[1] In this section, feature-test macros from all WG21 Technical Specifications are collected, for convenience.

[4.1] C++ Extensions for Library Fundamentals

[1] The recommended macro name is "__cpp_lib_experimental_" followed by the string in the "Macro Name Suffix" column.

Doc. No.	Title	Primary Section	Macro Name Suffix	Value	Header
N3915	apply() call a function with arguments from a tuple	3.2.2	apply	201402	<experimental/tuple>
N3932	Variable Templates For Type Traits	3.3.1	type_trait_variable_templates	201402	<experimental/type_traits>
N3866	Invocation type traits	3.3.2	invocation_type	201406	<experimental/type_traits>
N3916	Type-erased allocator for std::function	4.2	function_erased_allocator	201406	<experimental/functional>
N3905	Extending std::search to use Additional Searching Algorithms	4.3	boyer_moore_searching	201411	<experimental/functional>
N3672, N3793	A utility class to represent optional objects	5	optional	201411	<experimental/optional>
N3804	Any Library Proposal	6	any	201411	<experimental/any>
N3921	string_view: a non-owning reference to a string	7	string_view	201411	<experimental/string_view>
N3920	Extending shared_ptr to Support Arrays	8.2	shared_ptr_arrays	201406	<experimental/memory>
N3916	Polymorphic Memory Resources	8.4	memory_resources	201402	<experimental/memory_resource>
N3916	Type-erased allocator for std::promise	9.2	promise_erased_allocator	201406	<experimental/future>
N3916	Type-erased allocator for std::packaged_task	9.3	packaged_task_erased_allocator	201406	<experimental/future>
N3925	A sample Proposal	10.3	sample	201402	<experimental/algorithm>

[4.2] C++ Extensions for Parallelism

Name	Value	Header
__cpp_lib_experimental_parallel_algorithm	201505	<experimental/algorithm> <experimental/exception_list> <experimental/execution_policy> <experimental/numeric>

[4.3] File System Technical Specification

Name	Value	Header
__cpp_lib_experimental_filesystem	201406	<experimental/filesystem>

[4.4] C++ Extensions for Transactional Memory

Name	Value	Header
__cpp_transactional_memory	201505	*predeclared*
__has_cpp_attribute(optimize_for_synchronized)	201411 ?	*predeclared*

[5] Detailed explanation and rationale

[5.1] C++14 features

[1] Many of the examples here have been shamelessly and almost brainlessly plagiarized from the cited paper. Assistance with improving examples is solicited.

[5.1.1] N3323: A Proposal to Tweak Certain C++ Contextual Conversions

[1] This paper specifies a small change that is considered to be more of a bug fix than a new feature, so no macro is considered necessary.

[5.1.2] N3421: Making Operator Functors greater<>

[1] Example:

```
#if __cpp_lib_transparent_operators
        sort(v.begin(), v.end(), greater<>());
#else
        sort(v.begin(), v.end(), greater<valueType>());
#endif
```

[5.1.3] N3462: std::result_of and SFINAE

[1] The macro __cpp_lib_result_of_sfinae was originally erroneously recommended to be defined in the unrelated header <functional>. This error is being corrected, because it is believed that no implementation actually followed the erroneous recommendation.

[2] Example:

```
template<typename A>
#if __cpp_lib_result_of_sfinae
  typename std::result_of<inc(A)>::type
#else
  decltype(std::declval<inc>()(std::declval<A>()))
#endif
try_inc(A a);
```

[5.1.4] N3469: Constexpr Library Additions: chrono
N3470: Constexpr Library Additions: containers
N3471: Constexpr Library Additions: utilities

[1] These papers just add constexpr to the declarations of several dozen library functions in various headers. It is not clear that a macro to test for the presence of these changes would be sufficiently useful to be worthwhile.

[5.1.5] N3472: Binary Literals in the C++ Core Language

[1] Example:

```
int const packed_zero_to_three =
#if __cpp_binary_literals
        0b00011011;
#else
        0x1B;
#endif
```

[5.1.6] N3545: An Incremental Improvement to integral_constant

[1] Example:

```
constexpr bool arithmetic =
#if __cpp_lib_integral_constant_callable
```

```
        std::is_arithmetic<T>{}();
#else
        static_cast<bool>(std::is_arithmetic<T>{});
#endif
```

[5.1.7] N3624: Core Issue 1512: Pointer comparison vs qualification conversions

[1] This paper contained the wording changes to resolve a core issue. It did not introduce a new feature, so no macro is considered necessary.

[5.1.8] N3638: Return type deduction for normal functions

[1] This paper describes two separate features: the ability to deduce the return type of a function from the return statements contained in its body, and the ability to use `decltype(auto)`. These features can be implemented independently, so a macro is recommended for each.

[2] Examples:

```
template<typename T>
auto abs(T x)
#ifndef __cpp_return_type_deduction
        -> decltype(x < 0 ? -x : x)
#endif
{
        return x < 0 ? -x : x;
}
```

[5.1.9] N3639: Runtime-sized arrays with automatic storage duration

[1] This was removed from C++14 to TS 19569.

[2] Example:

```
#if __cpp_runtime_arrays
        T local_buffer[n];      // more efficient than vector
#else
        std::vector<T> local_buffer(n);
#endif
```

[5.1.10] N3642: User-defined Literals for Standard Library Types

[1] This paper specifies user-defined literal operators for two different standard library types, which could be implemented independently. Furthermore, user-defined literal operators are expected to be added later for at least one other library type. So for consistency and flexibility, each type is given its own macro.

[2] Examples:

[5.1.11] N3644: Null Forward Iterators

[1] Example:

[5.1.12] N3648: Wording Changes for Generalized Lambda-capture

[1] Example:

[5.1.13] N3649: Generic (Polymorphic) Lambda Expressions

[1] Example:

[5.1.14] N3651: Variable Templates

[1] Example:

[5.1.15] N3652: Relaxing constraints on constexpr functions / constexpr member functions and implicit const

[1] The major change proposed by this paper is considered to be strictly a further development of the `constexpr` feature of C++11. Consequently, the recommendation here is to give an increased value to the macro indicating C++11 support for `constexpr`.

[2] Example:

[5.1.16] N3653: Member initializers and aggregates

[1] Example:

[5.1.17] N3654: Quoted Strings Library Proposal

[1] Example:

[5.1.18] N3655: TransformationTraits Redux

[1] Example:

[5.1.19] N3656: make_unique

[1] Example:

[5.1.20] N3657: Adding heterogeneous comparison lookup to associative containers

[1] Example:

[5.1.21] N3658: Compile-time integer sequences

[1] Example:

[5.1.22] N3659: Shared locking in C++

[1] The original recommendation, of a macro defined in header `<mutex>`, was not useful, and has been retracted.

[2] For new headers, we have a long-term solution that uses `__has_include`. There was not sufficient support and a number of objections against adding macros to existing library header files, as there was not consensus on a place to put them.

[3] There is also a simple workaround for users that are not using libraries that define the header file: supplying their own header that is further down the search path than the library headers.

[4] Example:

```
#if __has_include(<shared_mutex>)
#include <shared_mutex>

// code that uses std::shared_mutex
#endif
```

[5.1.23] N3662: C++ Dynamic Arrays

[1] This was removed from C++14 to TS 19569.

[2] Example:

[5.1.24] N3664: Clarifying Memory Allocation

[1] The substantive change in this paper just relaxes a restriction on implementations. There is no new feature for a programmer to use, so no macro is considered necessary.

[5.1.25] N3667: Drafting for Core 1402

[1] This paper contained the wording changes to resolve a core issue. It did not introduce a new feature, so no macro is considered necessary.

[5.1.26] N3668: exchange() utility function

[1] Example:

[5.1.27] N3669: Fixing constexpr member functions without const

[1] This paper contained the wording changes to ensure that a minor change proposed by N3652 did not impact the standard library. It did not introduce a new feature, so no macro is considered necessary.

[5.1.28] N3670: Wording for Addressing Tuples by Type

[1] Example:

[5.1.29] N3671: Making non-modifying sequence operations more robust

[1] Example:

[5.1.30] N3672: A proposal to add a utility class to represent optional objects

[1] This was removed from C++14 to TS 19568.

[2] See N3659 for rationale.

[3] Example:

```
#if __has_include(<optional>)
#include <optional>

// code that uses std::optional
#endif
```

[5.1.31] N3673: C++ Library Working Group Ready Issues Bristol 2013

[1] This paper was just a collection of library issues. It did not introduce a new feature, so no macro is considered necessary.

[5.1.32] N3760: [[deprecated]] attribute

[1] Do we really need this here?

[2] Differentiating attribute availability based on compiler-specific macros is error- prone. Since vendors are allowed to provide implementation-specific attributes in addition to standards-based attributes, this feature provides a uniform, future-proof way to test the availability of attributes.

[3] Example:

```
#ifdef __has_cpp_attribute
#  if __has_cpp_attribute(deprecated)
#    define ATTR_DEPRECATED(msg) [[deprecated(msg)]]
#  else
#    define ATTR_DEPRECATED(msg)
#  endif
#endif

ATTR_DEPRECATED("f() has been deprecated") void f();
```

[5.1.33] N3776: Wording for ~future

[1] The change made by this paper is a bug fix, and no macro is considered necessary.

[5.1.34] N3778: C++ Sized Deallocation

[1] Example:

[5.1.35] N3779: User-defined Literals for std::complex

[1] Example:

[5.1.36] N3781: Single-Quotation-Mark as a Digit Separator

[1] Writing code that uses digit separators conditionally when they are available would require:

- either duplicating constants, presumably containing large numbers of digits, which would be error-prone and difficult to maintain,
- or using a scheme based on macros and token concatenation, which would effectively take the place of digit separators as supported by the language.

[2] Thus it is believed that a macro indicating support for digit separators in the language would not be sufficiently useful.

[5.1.37] N3786: Prohibiting "out of thin air" results in C++14

[1] The change made by this paper is a bug fix, and no macro is considered necessary.

[5.1.38] N3789: Constexpr Library Additions: functional

[1] See N3471 for rationale.

[5.1.39] N3887: Consistent Metafunction Aliases

[1] Example:

[5.1.40] N3891: A proposal to rename shared_mutex to shared_timed_mutex

[1] Example:

[5.1.41] N3910: What can signal handlers do? (CWG 1441)

[1] The change made by this paper is a bug fix, and no macro is considered necessary.

[5.1.42] N3924: Discouraging rand() in C++14

[1] The change made by this paper is not normative; no macro is necessary.

[5.1.43] N3927: Definition of Lock-Free

[1] The change made by this paper is a bug fix, and no macro is considered necessary.

[5.1.44] LWG issue 2112: User-defined classes that cannot be derived from

[1] Libraries that want to do empty-base optimization could reasonably want to use `!is_final && is_empty` if `is_final` exists, and fall back to just using `is_empty` otherwise.

[2] Example:

```
template<typename T>
struct use_empty_base_opt :
        std::integral_constant<bool,
```

```
                std::is_empty<T>::value
#if __cpp_lib_has_is_final
                && !std::is_final<T>::value
#endif
        >
{ };
```

[5.1.45] LWG issue 2247: Type traits and std::nullptr_t

[1] Example:

[5.1.46] LWG issue 2249: Remove gets from <cstdio>

[1] Since portable, well-written code does not use the `gets` function at all, there is no need for a macro to indicate whether it is available.

[5.1.47] LWG issue 2285: make_reverse_iterator

[1] Example:

[5.2] C++17 features

[5.2.1] N3911: TransformationTrait Alias void_t

[1] Example:

```
#if __cpp_lib_void_t
  using std::void_t;
#else
  template< class... >  using void_t = void;
#endif
```

[5.2.2] N3922: New Rules for auto deduction from braced-init-list

[1] This change is considered a fix to a problem in C++14, not a new feature, so no macro is recommended. Code that needs to work both with and without this change should not attempt to deduce a type from a direct *braced-init-list* initializer.

[5.2.3] N3928: Extending static_assert

[1] Example:

```
#if __cpp_static_assert
#if __cpp_static_assert > 201400
#define Static_Assert(cond) static_assert(cond)
#else
#define Static_Assert(cond) static_assert(cond, #cond)
#endif
#define Static_Assert_Msg(cond, msg) static_assert(cond, msg)
#else
#define Static_Assert(cond)
#define Static_Assert_Msg(cond, msg)
#endif
```

[5.2.4] N4051: Allow typename in a template template parameter

[1] This is a minor stylistic extension; there is no difference in functionality or verbosity between code that uses the `class` keyword and code that uses the `typename` keyword in this context. If portability is needed between implementations having and lacking this feature, generally it would be easier to continue to write code using `class` than to write and maintain conditional compilation to use `typename` when available. So a macro for this feature would not seem to be justified.

[5.2.5] N4086: Removing trigraphs??!

[1] This is a very low-level change, purely lexical. Writing code to use trigraphs when they are available and different characters when trigraphs are not available would be tedious, and the resulting code would be even uglier than code that just uses trigraphs.

[2] Even in circumstances where use of a character not in the ISO/IEC 646 Basic Character Set is problematic, it would generally be easier to use alternative tokens (a.k.a. “digraphs”) than to conditionally use trigraphs.

[5.2.6] N4089: Safe conversions in unique_ptr<T[]>

[1] This is considered a fix for a library issue, to remove an unnecessary restriction; no macro is considered necessary.

[5.2.7] N4169: A proposal to add invoke function template

[1] Example:

```
template<typename F>
auto deref_fn(F&& f)
{
  return [f](auto&&... args) {
#if __cpp_lib_invoke
    return *std::invoke(f, std::forward<decltype(args)>(args)...);
```

```
#else
    return *f(std::forward<decltype(args)>(args)...);
#endif
  };
}
```

[2] In this example, member function pointers are supported only if `invoke` is available.

[5.2.8] N4190: Removing auto_ptr, random_shuffle(), And Old <functional> Stuff

[1] These library features are removed because superior alternatives to them were introduced in C++11. Because these alternatives are superior, there is little motivation to maintain code that uses one of these obsolescent features when available.

[5.2.9] N4230: Nested namespace definition

[1] This feature doesn't provide any new functionality; it just makes it somewhat easier to write code. For code that needs to be portable to an implementation that doesn't provide this feature, it would be easier just to avoid using the feature than it would be maintain the code to use it when available but not otherwise. Therefore, a macro to indicate the availability of this feature would not seem to be justified.

[5.2.10] N4258: Cleaning-up noexcept in the Library

[1] Example:

```
template <class T, class Allocator = std::allocator<T>>
class myContainer {
    typedef std::allocator_traits<Allocator> alloc_traits;
public:
    ...
    myContainer& operator=(myContainer&& x)
#ifdef __cpp_lib_allocator_traits_is_always_equal
      noexcept(alloc_traits::propagate_on_container_move_assignment::value ||
               alloc_traits::is_always_equal::value);
#else
      noexcept(alloc_traits::propagate_on_container_move_assignment::value);
#endif
    ...
};
```

[2] Without the trait, the code will be correct, but container move-assignment would have fewer exception guarantees, especially with pre-C++11 stateless allocators. An implementation of a mutating algorithm might choose to use copy-assignment instead of move-assignment if the `noexcept` clause evaluates to false, because the implementation is unable to determine whether the allocator always compares equal.

[5.2.11] N4259: Wording for std::uncaught_exceptions

[1] Example:

```
bool throw_may_terminate()
{
#if __cpp_lib_uncaught_exceptions
    return std::uncaught_exceptions() > 0;
#else
    return std::uncaught_exception();
#endif
}
```

[5.2.12] N4261: Proposed resolution for Core Issue 330: Qualification conversions and pointers to arrays of pointers

[1] This paper fixes a core language issue; no new feature is introduced.

[5.2.13] N4262: Wording for Forwarding References

[1] The changes detailed in this paper are solely editorial. It does not introduce any new feature.

[5.2.14] N4266: Attributes for namespaces and enumerators

[1] Example:

```
enum {
old_val
#if __cpp_enumerator_attributes
       [[deprecated]]
#endif
, new_val };
```

[5.2.15] N4267: Adding u8 character literals

[1] This feature gives guaranteed access to the Unicode encoding for a 7-bit ASCII character even in an implementation that uses a different character encoding. In an application that needs this functionality, it would probably be easier simply to hard-code the numeric values of the needed encodings. Thus a macro for this feature would seem to be of limited usefulness.

[5.2.16] N4268: Allow constant evaluation for all non-type template arguments

[1] Example:

```
int n = 0, m = 0;
int &r = n;
template<int &> struct X {};
#if __cpp_nontype_template_args
  X<r> x;
#else
  std::conditional<&r == &n, X<n>, X<m>>::type x;
#endif
```

[5.2.17] N4277: TriviallyCopyable reference_wrapper

[1] This paper imposes a new requirement on implementations, which most implementations had already satisfied. A reliable indication of when this requirement was not satisfied would have been useful for programming; an indication that it is satisfied would not be so useful.

[5.2.18] N4279: Improved insertion interface for unique-key maps

[1] Example:

```
#if __cpp_lib_map_try_emplace
    m.insert_or_assign("foo", std::move(p));
#else
    m["foo"] = std::move(p);
#endif
```

[2] In this example, if `insert_or_assign` is not available, the type of `p` must be default-constructible.

[5.2.19] N4280: Non-member size() and more

[1] Example:

```
auto n =
#if __cpp_lib_nonmember_container_access
    std::size(c);
#else
    c.size();
#endif
```

[2] In this example, an array type is supported for `c` only if the nonmember `size` function is available.

[5.2.20] N4284: Contiguous Iterators

[1] The changes detailed in this paper are solely editorial. It does not introduce any new feature.

[5.2.21] N4285: Cleanup for exception-specification and throw-expression

[1] The changes detailed in this paper are solely editorial. It does not introduce any new feature.

[5.2.22] N4295: Folding expressions

[1] Example:

```
#if __cpp_fold_expressions
template<typename... T>
  auto sum(T... args)  { return (args + ...); }
#else
auto sum() { return 0; }
template<typename T>
  auto sum(T t) { return t; }
template(typename T, typename... Ts)
  auto sum(T t, Ts... ts) { return t + sum(ts...); }
#endif
```

[5.2.23] N4366: LWG 2228 missing SFINAE rule

[1] This change fixes a defect in the library. No new feature is added, so no feature-test macro is needed.

[5.2.24] N4387: Improving pair and tuple

[1] This paper resolves some library issues. There is no new feature; no macro is required. Someone please confirm my guess here.

[5.2.25] N4389: Wording for bool_constant

[1] Example:

```
#if __cpp_lib_bool_constant
  template <bool B>
  using my_bool_constant = std::bool_constant<B>;
#else
  template <bool B>
  using my_bool_constant = std::integral_constant<bool, B>;
#endif
```

[5.2.26] N4508: A proposal to add shared_mutex (untimed)

[1] Example:

[2] Lenexa

[5.2.27] N4510: Minimal incomplete type support for standard containers

[1] Example:

[2] Lenexa

[5.2.28] P0001R1: Remove Deprecated Use of the register Keyword

[1] The simplest way to write code to be portable between implementations that have and lack this change is to avoid using the `register` keyword. A feature-test macro would not be sufficiently useful.

[5.2.29] P0002R1: Remove Deprecated operator++(bool)

[1] The simplest way to write code to be portable between implementations that have and lack this change is to avoid applying `++` to `bool` objects. A feature-test macro would not be sufficiently useful.

[5.2.30] P0004R1: Remove Deprecated iostreams aliases

[1] The simplest way to write code to be portable between implementations that have and lack this change is not to use the previously deprecated features. A feature-test macro would not be sufficiently useful.

[5.2.31] P0006R0: Adopt Type Traits Variable Templates from Library Fundamentals TS for C++17

[1] Example:

[2] Kona

[5.2.32] P0007R1: Constant View: A proposal for a std::as_const helper function template

[1] Example:

[2] Kona

[5.2.33] P0012R1: Make exception specifications be part of the type system

[1] Example:

[2] Kona

[5.2.34] P0013R1: Logical Operator Type Traits

[1] Example:

```
template<typename... T>
  using condition =
#if __cpp_lib_logical_traits
  conjunction<is_pointer<decay_t<T>>...>;
#else
  bool_constant<(is_pointer<decay_t<T>>::value && ...)>;
#endif

template<typename... T>
  enable_if_t<condition<T...>::value>
  foo(T&&...) { }

template<typename... T>
  enable_if_t<!condition<T...>::value>
  foo(T&&...) { }
```

[2] Using conjunction is more efficient at compile-time, because it short-circuits and doesn't evaluate every expression in the pack expansion if it doesn't need to.

[5.2.35] P0061R1: __has_include for C++17

[1] The presence of this feature can be determined by testing whether the `__has_include` macro is defined. However, because it is a function-like macro, it has no specific value that can be tested.

[5.2.36] P0074R0: Making std::owner_less more flexible

[1] Example:

```
#if __cpp_lib_transparent_operators >= 201510
using generic_owner_less = std::owner_less<>;
#else
```

```
struct generic_owner_less {
  template<typename T, typename U>
    bool
    operator()(const std::shared_ptr<T>& p1,
               const std::shared_ptr<U>& p2) const
    { return p1.owner_before(p2); }
};
#endif
```

[5.2.37] P0092R1: Polishing <chrono>

[1] Example:

[2] Kona

[5.2.38] P0134R0: Introducing a name for brace-or-equal-initializers for non-static data members

[1] This change is entirely editorial; no feature is added, and no test macro is needed.

[5.2.39] P0136R1: Rewording inheriting constructors (core issue 1941 et al)

[1] This change is considered a minor tweak to the inheriting constructors feature, and mostly just fixes issues with the original description. However, it can change the behavior of some code. Therefore the value of the `__cpp_inheriting_constructors` macro is adjusted.

[2] Example:

[3] Kona

[5.2.40] P0156R0: Variadic lock_guard

[1] In a previous revision, it was recommended that a test macro for this feature be defined in `<thread>`. That was a mistake; the relevant code is actually in `<mutex>`.

[2] This feature was subsequently removed from C++17. The functionality it had provided is now provided by a `scoped_lock`.

[5.2.41] P0005R4: Adopt not_fn from Library Fundamentals 2 for C++17

Example:

Jacksonville

[5.2.42] P0017R1: Extension to aggregate initialization

Example:

Jacksonville

[5.2.43] P0018R3: Lambda Capture of *this by Value as [=,*this]

Example:

Jacksonville

[5.2.44] P0024R2: The Parallelism TS Should be Standardized

Example:

Jacksonville

[5.2.45] P0025R0: An algorithm to "clamp" a value between a pair of boundary values

Example:

Jacksonville

[5.2.46] P0030R1: Proposal to Introduce a 3-Argument Overload to std::hypot

Example:

Jacksonville

[5.2.47] P0031R0: A Proposal to Add Constexpr Modifiers to reverse_iterator, move_iterator, array and Range Access

Jacksonville

Example:

[5.2.48] P0033R1: Re-enabling shared_from_this

Example:

```
template<class ObjectType>
void register_observers(ObjectType& obj)
{
#if __cpp_lib_enable_shared_from_this
   auto wptr = obj.weak_from_this();
#else
   auto sptr = obj.shared_from_this();
   // Fails if sptr is not std::shared_ptr
   // (e.g. std::experimental::shared_ptr or boost::shared_ptr)
   auto wptr = weak_ptr<decltype(sptr)::element_type>(sptr);
   sptr.reset(); // drop the strong reference as soon as possible
#endif
   register_observer_1(wptr);
   register_observer_2(wptr);
}
```

[5.2.49] P0036R0: Unary Folds and Empty Parameter Packs

This is a change to the original definition of the “unary fold” feature. The presence or absence of this change should be distinguished by the value of the `__cpp_fold_expression` macro.

[5.2.50] P0077R2: is_callable, the missing INVOKE related trait

Example:

Jacksonville

[5.2.51] P0138R2: Construction Rules for enum class Values

This doesn't provide any new functionality; it just makes some kinds of code simpler to write. Code that needs to be portable to implementations lacking this feature would be still more complicated if it tried to use the new feature when available. So a macro for this feature would not seem to be justified.

[5.2.52] P0152R1: constexpr atomic<T>::is_always_lock_free

Example:

Jacksonville

[5.2.53] P0154R1: constexpr std::hardware_{constructive,destructive}_interference_size

Example:

Jacksonville

[5.2.54] P0170R1: Wording for Constexpr Lambda

The changes proposed in this paper are considered to be an extension of the capabilities of the `constexpr` keyword. Therefore the presence or absence of this feature should be distinguished by the value of the `__cpp_constexpr` macro.

[5.2.55] P0184R0: Generalizing the Range-Based For Loop

This is considered an extension of the range-based `for` statement. Therefore the presence or absence of this feature should be distinguished by the value of the `__cpp_range_based_for` macro.

[5.2.56] P0185R1: Adding [nothrow-]swappable traits

Example:

```
template<class T>
  void my_swap(T& x, T& y)
#if __cpp_lib_is_swappable
  noexcept(std::is_swappable_v<T>); // Covers all Lvalue Swappable cases
#else
  noexcept(
    std::is_nothrow_move_constructible_v<T> &&
    std::is_nothrow_move_assignable_v<T>
  );
#endif
```

[5.2.57] P0218R1: Adopt the File System TS for C++17

Example:

Jacksonville

[5.2.58] P0220R1: Adopt Library Fundamentals V1 TS Components for C++17

[5.2.59] P0226R1: Mathematical Special Functions for C++17

Example:

Jacksonville

[5.2.60] P0245R1: Hexadecimal floating literals for C++

Example:

Jacksonville

[5.2.61] P0253R1: Fixing a design mistake in the searchers interface in Library Fundamentals

This is a fix that is applied to the searchers interface at the same time that it is incorporated into the standard. The unfixed state exists only in the TS, which has its own macro name and value. The new macro for the searchers interface in the standard, with its new value, will be enough to indicate that this fix is applied.

[5.2.62] P0272R1: Give 'std::string' a non-const '.data()' member function

This doesn't provide any new functionality; it just makes some kinds of code simpler to write. Code that needs to be portable to implementations lacking this feature would be still more complicated if it tried to use the new feature when available. So a macro for this feature would not seem to be justified.

[5.2.63] LWG2296: std::addressof should be constexpr

Example:

```
struct optional {
  bool b;
  T t;
  constexpr T *get() {
    return b ?
#if __cpp_lib_addressof_constexpr
      std::addressof(t)
#else
      &t
#endif
      : nullptr;
  }
};
```

It is more important for `get` to be `constexpr` than for it to work if there is an overloaded `operator&`, so `addressof` is used only if it would actually work in a constant expression.

Oulu:

[5.2.64] P0028R4: Using attribute namespaces without repetition

[5.2.65] P0032R3: Homogeneous interface for variant, any and optional

[5.2.66] P0035R4: Dynamic memory allocation for over-aligned data

Example:

[5.2.67] P0040R3: Extending memory management tools

[5.2.68] P0063R3: C++17 should refer to C11 instead of C99

This paper is primarily editorial. There are a few additional C library functions that are formally required as a result of these changes, but in many cases they are not under the control of the C++ library implementer. It might be possible to tell the difference by the value of the `__STDC__` macro. It's not clear that any real purpose would be served by recommending a C++-specific macro to distinguish the difference.

[5.2.69] P0083R3: Splicing Maps and Sets

Example:

```
void update(std::set<X>& set, const X& elem, int val)
{
  auto pos = set.find(elem);
  if (pos == set.end())
    return;
#if __cpp_lib_node_extract
  auto next = std::next(pos);
  auto x = set.extract(pos);
  x.value().update(val);
  set.insert(next, std::move(x));
#else
  X tmp = *pos;
  pos = set.erase(pos);
  tmp.update(val);
  set.insert(pos, std::move(tmp));
#endif
```

```
}
```

The version using `extract` doesn't need to copy or move any `X` objects and performs no memory allocation.

[5.2.70] P0084R2: Emplace Return Type

Code that needs to be portable between implementations with and without changes required by this paper should simply assume the previous rules.

[5.2.71] P0088R3: Variant: a type-safe union for C++17

[5.2.72] P0091R3: Template argument deduction for class templates

[5.2.73] P0127R2: Declaring non-type template parameters with auto

Example:

[5.2.74] P0135R1: Wording for guaranteed copy elision through simplified value categories

This feature doesn't provide any new functionality; it just makes it somewhat easier to write code. For code that needs to be portable to an implementation that doesn't provide this feature, it would be easier just to avoid using the feature than it would be maintain the code to use it when available but not otherwise. Therefore, a macro to indicate the availability of this feature would not seem to be justified.

[5.2.75] P0137R1: Core Issue 1776: Replacement of class objects containing reference members

Example:

[5.2.76] P0145R3: Refining Expression Evaluation Order for Idiomatic C++

Code that needs to be portable between implementations with and without changes required by this paper should simply assume the previous rules.

[5.2.77] P0163R0: shared_ptr::weak_type

Example:

```
template<class ObjectType>
void register_observers(ObjectType& obj)
{
    auto sptr = obj.shared_from_this();
#if __cpp_lib_shared_ptr_weak_type
    decltype(sptr)::weak_type wptr(sptr);
#else
    // Fails if sptr is not std::shared_ptr
    // (e.g. std::experimental::shared_ptr or boost::shared_ptr)
    auto wptr = weak_ptr<decltype(sptr)::element_type>(sptr);
#endif
    sptr.reset(); // drop the strong reference as soon as possible
    register_observer_1(wptr);
    register_observer_2(wptr);
}
```

[5.2.78] P0174R2: Deprecating Vestigial Library Parts in C++17

This paper deprecates some library facilities for which superior alternatives exist. Code that needs to be portable should use the alternatives; a macro to enable code to adapt would not be useful.

[5.2.79] P0175R1: Synopses for the C library

The changes proposed in this paper are entirely editorial; no macro is required.

[5.2.80] P0180R2: Reserve a New Library Namespace Future Standardization

This paper does not describe a new feature. It describes a retracted feature: the ability of a program to define a namespace with any of a certain set of names. Portable code should avoid defining such a namespace; no macro is necessary.

[5.2.81] P0181R1: Ordered By Default

Example:

[5.2.82] P0209R2: make_from_tuple: apply for construction

Example:

[5.2.83] P0217R3: Proposed wording for structured bindings

Example:

[5.2.84] P0219R1: Relative Paths for Filesystem

[5.2.85] P0254R2: Integrating std::string_view and std::string

[5.2.86] P0258R2: has_unique_object_representations - wording

Example:

[5.2.87] P0283R2: Standard and non-standard attributes

[5.2.88] P0292R2: constexpr if: A slightly different syntax

Example:

[5.2.89] P0295R0: Adopt Selected Library Fundamentals V2 Components for C++17

Example:

[5.2.90] P0296R2: Forward progress guarantees: Base definitions
P0299R1: Forward progress guarantees for the Parallelism TS features

These papers effectively add only definitions. The added guarantees are intended only to formalize widespread existing practice.

[5.2.91] P0302R1: Removing Allocator Support in std::function

Since these constructors could not be used portably or reliably, there is no need for a feature test macro to indicate their absence.

[5.2.92] P0305R1: Selection statements with initializer

This feature doesn't provide any new functionality; it just makes it somewhat easier to write code. For code that needs to be portable to an implementation that doesn't provide this feature, it would be easier just to avoid using the feature than it would be maintain the code to use it when available but not otherwise. Therefore, a macro to indicate the availability of this feature would not seem to be justified.

[5.2.93] P0307R2: Making Optional Greater Equal Again

[5.2.94] P0336R1: Better Names for Parallel Execution Policies in C++17

[5.2.95] P0337R0: Delete operator= for polymorphic_allocator

This paper just fixes a defect. It doesn't add any new feature, so no macro is needed.

[5.2.96] P0346R1: A <random> Nomenclature Tweak

The paper changes nothing but terminology, and is entirely editorial; no macro is needed.

[5.2.97] P0358R1: Fixes for not_fn

The changes in this paper were adopted into the working draft of the standard at the same time as the `not_fn` feature they modify. The specification in the technical specification never appeared as such in any working draft of the standard, so a macro to distinguish the presence or absence of these changes is not considered necessary.

[5.2.98] P0371R1: Temporarily discourage memory_order_consume

This paper is entirely editorial; no macro is required.

[5.2.99] P0386R2: Inline Variables

Example:

```
#if __cpp_inline_variables >= 201703
#define INLINE_VAR_DECL inline
#else
#define INLINE_VAR_DECL /*fingers crossed*/
#endif

INLINE_VAR_DECL constexpr piecewise_construct_t piecewise_construct{};
```

Do we really want to (appear to) recommend that people write code like this?

[5.2.100] P0391R0: Introducing the term "templated entity"

This paper is entirely editorial, introducing a new term to make it easier to correctly draft the standard; no macro is necessary.

[5.2.101] P0392R0: Adapting string_view by filesystem paths

[5.2.102] P0393R3: Making Variant Greater Equal

The changes in this paper were adopted into the working draft of the standard at the same time as the `variant` feature they modify. The specification in P0083R3 never appeared as such in any working draft, so a macro to distinguish the presence or absence of these changes is not considered necessary.

[5.2.103] P0394R4: Hotel Parallelifornia: terminate() for Parallel Algorithms Exception Handling

[5.2.104] P0398R0: Core issue 1518: Explicit default constructors and copy-list-initialization

This paper describes a bug fix, not a new feature; no new macro is needed.

Issaquah:

[5.2.105] P0003R5: Removing Deprecated Exception Specifications from C++17

[5.2.106] P0067R5: Elementary string conversions

[5.2.107] P0195R2: Pack expansions in using-declarations

```
#if __cpp_variadic_using >= 201611
template<typename ...T> struct Callable : T... {
  using T::operator() ...;
};
#else
template<typename ...T> struct Callable;
template<typename T, typename ...U> struct Callable<T, U...> : T,
Callable<U...> {
  using T::operator();
  using Callable<U...>::operator();
};
template<typename T> struct Callable<T> : T {
  using T::operator();
};
template<> struct Callable<> {};
#endif
```

[5.2.108] P0497R0: Fixes to shared_ptr support for arrays

[5.2.109] P0505R0: Wording for GB 50

[5.2.110] P0512R0: Class Template Argument Deduction Assorted NB resolution and issues

[5.2.111] P0522R0: Matching of template template-arguments excludes compatible templates

---

Kona:

[5.2.112] P0156R2: Variadic lock_guard

[5.2.113] P0298R3: A byte type definition

[5.2.114] P0317R1: Directory Entry Caching for Filesystem

[5.2.115] P0604R0: Resolving GB 55, US 84, US 85, US 86

Example:

```
#if __cpp_lib_is_invocable >= 201703
using std::is_invocable;
#elif __cpp_lib_void_t >= 201411
template<typename R, typename = void>
  struct is_invocable_impl : std::false_type
  { };
template<typename R>
  struct is_invocable_impl<R, std::void_t<typename R::type>> : std::true_type
  { };
template<typename F, typename... Args>
  struct is_invocable : is_invocable_impl<std::result_of<F&&(Args&&...)>>::type
  { };
#else
#error either is_invocable or void_t is needed
#endif
```

[5.2.116] P0618R0: Deprecating <codecvt>

[5.2.117] LWG2911: An is_aggregate type trait is needed

```
#include <vector>
template<typename T, typename... Args>
T make(Args&&... args)
{
#if __cpp_lib_is_aggregate
    if constexpr (std::is_aggregate_v<T>)
        return { std::forward<Args>(args)... };
    else
#endif
        return T(std::forward<Args>(args)...);
}
```

```
struct Agg { int i; };
int main()
{
    auto v = make<std::vector<int>>(1, 2);
#if __cpp_lib_is_aggregate
    // make<> only supports aggregates if std::is_aggregate is available
    auto a = make<Agg>(1);
#endif
}
```

[6] Annex: Model wording for a Technical Specification

[1] It is recommended that Technical Specifications include the wording below. *yyyymm* indicates the year and month of the date of the TS.

1.x Feature testing [intro.features]

An implementation that provides support for this Technical Specification shall define the feature test macro(s) in Table *X*.

Table X — Feature Test Macro(s)

Name	Value	Header
__cpp_*name*	*yyyymm*	"*predefined*" **OR** "<*name*>"

[7] Revision history

Date	Document	Description
2017-07-26	P0096R4	Added C++17 features from Oulu, Issaquah and Kona
2016-03-08	SD-6 P0096R3	Updated from P0096R2
2016-02-23	P0096R2	Fixed recommendation for P0074R0 A few editorial fixes
2016-01-19	P0096R1	Added C++17 features from Kona
2015-09-16	P0096R0	Added C++17 features from Lenexa A few adjustments to C++17 recommendations Added model wording for a Technical Specification Added summaries of recommendations from TS'es
2015-04-09	N4440	Added C++17 features from Urbana A couple of changes to C++14 recommendations A few new C++11 features
2014-12-29	SD-6	Updated from N4200
2014-10-08	N4200	Updated for final C++14 Added more support for C++11 and C++98
2014-05-22	N4030	Updated for changes from Chicago and Issaquah meetings Added `__has_cpp_attribute` test
2013-11-27	SD-6	Initial publication No substantive changes from N3745
2013-08-28	N3745	Endorsed by WG21 membership
2013-06-27	N3694	Initial draft